

CSEN 601: Computer System Architecture

Spring 2018

Project Description

Project Name: Pipelined MIPS datapath simulator

Grading weight: This project and lab assessments accounts for 15% of the course mark. A bonus up to 5% **of the project mark** will be given for implementing **one** of the *bonus* features suggested below.

Project Overview: The goal of this project is to implement a low-level cycle-accurate pipelined MIPS datapath simulator. Simulating the datapath includes simulating all of its storage components (register file, memories, and pipeline registers) and all of its control signals. This document details the supported instructions, the inputs to the simulator, and the expected outputs.

You should choose to implement the simulator using one of the following methods according to your major:

- **MET students** should implement the project using **Verilog**.
- **Networks students** can choose one of the following:
 - **Logisim**
 - **Verilog**

Team Size: 1 to 6 students (preferably 2 or more).

Important Plagiarism notice: You have to write your own code/design your own circuit from scratch. Projects based on others will receive a grade of **zero** in the entire project (even if it is heavily re-factored/modified, etc...). Examples of such sources include (but are not limited to): other teams, previous year projects, open-source software, tutors, etc...

Project Deadline: Friday ***April 21st***, 2016. Evaluations will take place immediately afterwards.

Instruction set architecture (ISA): The simulator must support the following MIPS instructions **only**:

- Arithmetic: add, sub, addi
- Load/Store: lw, sw, lh, lhu
- Logic: and, or, sll, srl, and, or
- Control flow: beq
- Comparison: slt, sltu

Common Notes:

- The simulator should assume a pipelined datapath identical to the one presented in the lectures. The pipeline consists of the same five stages discussed in lectures (ID, IF, EXE, MEM and WB).
- Note that, this datapath **does not** detect or handle hazards at all, which means that the input program has to be hazard-free to produce correct results.
- The register file consists of the 32 registers including the \$zero register. Register 0 always contains the value 0. You will need to make sure you initialize register 0 appropriately and make sure any attempt to modify it does not succeed.

Version 1: Using Verilog

Important Notes:

- On a Verilog simulator like Verilogger or ModelSim Altera, you should simulate the program execution showing the contents (decimal and/or hexadecimal) of all the fields of the pipeline registers (PC, IF/ID, ID/EX, EX/MEM, and MEM/WB) including control signals at **each clock cycle**.
- Any control signals that are not part of the pipeline registers (e.g., the PCSrc) should also be displayed at each clock cycle.
- On the simulator, you should also keep track of the **register file contents** and **memory contents**.
- Finally, you should keep track of the number of clock cycles spanned during the execution of the input program.

Testing:

- *Assembly program:* The test bench should load a program (and data if any) into the memory (using the supported instructions only). The program can be written directly on an external assembler like **MARS**, and copy the machine code into your test bench.
- *Tracking Cycles:* It's preferable that you show the user the outputs of each cycle continuously.

Example:

Cycle 1: Instruction 1 fetched.

Cycle 2: Instruction 1 decoded, Instruction 2 fetched.

Cycle 3: Instruction 1 executed, Instruction 2 decoded, Instruction 3 fetched.

- *Outputs:* At the end of the program, you should print out to the user the content of both the register file as well as the memory.

Version 2: Using the LOGISIM logic simulator**Important notes:**

- You are required to implement a pipelined MIPS processor on the LOGISIM logic simulator.
- The largest supported memory size in Logisim is 16Mx32 (i.e. it has 24 address lines). That is why you will be using that memory, and you should adjust your design accordingly. The memory in Logisim is word addressable (and not byte-addressable as we are studying in the course). Thus, you need to adjust your design accordingly. For instance, the PC needs to get incremented by 1 instead of 4.
- The whole project and the different modules should be implemented as **one** .circ file.

Testing:

- In Logisim, load the Program memory with a machine code program (you may use MARS to generate the machine code for any given assembly program.)
- Ensure the clock is at 0; if it is at 1, select the Poke Tool (the hand icon) and click the clock in the bottom left corner to bring it back to 0.

- With the Poke Tool (the hand icon) selected, click the clear button in the circuit's bottom left corner to reset the PC and all registers to 0.
- To execute the program, you can repeatedly click the clock using the Poke Tool to step one half-cycle at a time, or you can select "Ticks Enabled" from the Simulate menu to start the clock running automatically through the program. (You can slow its speed through the "Tick Frequency" submenu.)
- To view current register values, right-click the "regs" circuit and select "View registers". You can double-click "main" to go back to the overall circuit.

Bonus Feature:

- Handling Hazards as follows:
 - Forwarding (ALU to ALU and MEM to ALU)
 - Branch prediction (predict branch as not taken)